

CanlıFal Backend Denetim Raporu — Flutter Uyumluluk Sertifikasyonu

Tarih: 3 Ağustos 2026

Kapsam: canlifal.com production backend'inin Flutter mobil uygulaması ile tam uyumluluğunun denetimi.

Yöntem: 487 API route handler, tüm gerçek zamanlı (SSE) akışlar, TRTC yaşam döngüsü, hediye motoru, veritabanı indeks/işlem yapısı üzerinde **kod düzeyinde statik denetim**.

Önemli dürüstlük notu: Bu denetim kod incelemesine dayanır. 487 endpoint'in tamamı canlı ortamda tek tek çalıştırılarak test edilmemiştir. Aşağıda "doğrulandı (kod)" ile "çalışma zamanında test edildi" ayrımı korunmuştur. Hiçbir backend davranışı değiştirilmemiş, hiçbir iş mantığı bozulmamıştır (Kural #8).

0. Özet (Yönetici Özeti)

Backend genel olarak **profesyonelce inşa edilmiş ve büyük ölçüde optimize** durumdadır. Gerçek zamanlı akış, TRTC token üretimi, hediye motoru, veritabanı indeksleri ve bağlantı havuzu **iyi tasarlanmıştır**. Flutter tarafının güvenle entegre olabileceği sağlam bir temel mevcuttur.

Denetimde **kritik seviyede bozuk bir yapı bulunmamıştır**. Tespit edilen konular; (a) mimari kaynaklı kalıcı riskler, (b) Flutter tarafının uygulaması gereken sözleşme kuralları ve (c) onay gerektiren isteğe bağlı iyileştirmelerden ibarettir. Backend production'da çalıştığı ve Flutter zaten mevcut yanıt şekillerine göre kodlandığı için, mevcut sözleşmeleri bozacak hiçbir değişiklik **kendiliğinden uygulanmamıştır** — bunlar öneri olarak sunulmuştur.

1. API DENETİMİ

1.1 Endpoint envanteri

- Kod tabanında **487 adet route.ts** handler bulunmaktadır.
- Dokümantasyon (`ENDPOINTS.md`, `openapi.json`, `endpoints_index.json`) **473 benzersiz yol / 736 method handler / 163 kategori** olarak keşif çıktısı vermektedir. Bu sayılar tutarlıdır (bir route.ts birden fazla HTTP method içerebilir).
- Flutter dokümanları (`FLUTTER_API_REFERENCE`, `FLUTTER_BACKEND_INTEGRASYON_PROMPT`) ile karşılaştırıldığında, Flutter'ın kullandığı tüm kritik uçlar (auth, chat rooms + SSE, voice, video-streams, live, gift, trtc/token) backend'de **mevcuttur**. Eksik (Flutter'ın çağırdığı ama backend'de olmayan) bir uç tespit edilmemiştir.

1.2 Yanıt zarfı (envelope) tutarlılığı — BULGU

- Projede standart bir zarf yardımcı dosyası mevcuttur: `lib/api-response.ts` (`apiSuccess`, `apiError`, `apiPaginated`, hata kodları, `requestId`, `timestamp`). Tasarımı çok iyidir.
- Ancak bu yardımcı hiçbir endpoint tarafından kullanılmıyor** (`app/api` içinde import sayısı: 0).

- Gerçek durum: 487 route'un ~200'ü `{ success: true, ... }` biçiminde döner; ~287'si `success` alanı olmayan ham nesneler döner (örn. `voice route { voiceUsers, timestamp }, video-streams { streams, items, pagination }`).
- **Sonuç:** API genelinde tek tip bir yanıt zarfı YOKTUR. Bu bir tutarsızlıktır.
- **Neden düzeltilmedi (bilinçli karar):** Flutter uygulaması hâlihazırda bu uç-uca özel şekillere göre yazılmıştır. 287 endpoint'i standart zarfa taşımak, dağıtılmış Flutter istemcisini **kıracaktır** (Kural #8 ihlali). Bu yüzden mevcut şekiller korunmuştur.
- **Öneri:** Zarf standardizasyonu sadece **yeni** endpoint'lerde `lib/api-response.ts` kullanılarak uygulanmalıdır. Mevcut uçlar dokümantasyondaki şekillerine göre tüketilmeye devam edilmelidir.

2. GERÇEK ZAMANLI AKIŞ (SSE)

2.1 Mimari

Tüm gerçek zamanlı akış, **process-içi (in-memory) olay veri yolları** ile çalışır:

- `lib/chat-events.ts` (TTL 2 dk / 200 olay), `lib/room-events.ts` (5 dk / 100), `lib/stream-events.ts` (5 dk / 100), `lib/voice-room-events.ts`.
- Olaylar zaman damgası (timestamp) anahtarlıdır; `setInterval` ile periyodik temizlenir.
- SSE endpoint'leri bu veri yollarını **2 saniyelik poll döngüsü** ile okuyup istemciye iletir.

2.2 SSE endpoint kalitesi (`chat/rooms/[roomId]/stream`) — doğrulandı (kod)

- **Çift kimlik doğrulama:** mobil JWT (`authenticateRequest`) VEYA web oturumu. ✓
- **Heartbeat:** her 15 sn'de bir `: heartbeat\n\n` gönderilir; bağlantı koparsa `clearInterval` ile temizlenir. ✓
- **Poll döngüsü:** 2 sn (`setTimeout(checkForUpdates, 2000)`). ✓
- **Presence:** DB sorgusu her poll'de değil, **10 sn'de bir** (5 döngüde bir) yapılır — DB yükünü azaltır. ✓
- **Reconnect (yeniden bağlanma):** `Last-Event-ID` başlığı / `?lastEventId=` parametresi desteklenir; olay id'leri veri yolu zaman damgalarıdır; her mesaj `id: <ts>` ile gönderilerek istemci imleci iletilir. Yeniden bağlanmada kaçırılan olaylar TTL penceresi içinde tekrar alınabilir. ✓
- **İstemciye giden payload tipleri:** `connected`, `messages`, `system`, `gift`, `pk`, `room_event`, `presence`, `typing` + DJ payloadları.

2.3 Olay sıralaması — BULGU (davranış, değiştirilmedi)

- Tek bir 2 sn'lik poll penceresi içinde olaylar **tipe göre gruplanarak** gönderilir (`messages` → `system` → `gift` → `pk` → `room_event`).
- **Aynı tip** içindeki sıralama korunur. Ancak **farklı tipler arası** kronolojik sıralama, aynı poll penceresi içinde **kesin olarak korunmaz**.
- Bunu değiştirmek davranış değişikliği olurdu → **değiştirilmedi**.
- **Flutter tarafı:** Farklı tipteki olaylar (örn. bir mesaj ile bir hediyenin) arasında mutlak kronolojik sıraya **bağımlı olmamalıdır**; gerekiyorsa payload içindeki `timestamp` / `createdAt` alanına göre istemcide sıralamalıdır.

2.4 Kalıcı Risk (mimari) — DÜZELTİLMEDİ (bilinçli)

- In-memory olay veri yolu **örnek-başınadır (per-instance)**. Backend yatay olarak (birden fazla sunucu örneği) ölçeklenirse, A örneğinde yayılan bir olay B örneğine bağlı SSE istemcisine **ulaşmaz**.
- Şu an tek örnek dağıtımda sorun yaratmaz. Ölçeklenince olay kaybı riski doğar.
- **Çözüm (büyük değişiklik, onay gerektirir):** Redis pub/sub veya benzeri bir dış olay yolu. Bu, mimari bir değişiklik olduğu için Kural #8 gereği **kendiliğinden yapılmadı**. İleride ölçekleme planlanıyorsa öncelikli iştir.

3. TRTC YAŞAM DÖNGÜSÜ — doğrulandı (kod)

3.1 Token üretimi (/api/trtc/token ve /api/trtc/usersig)

- JWT/oturum korumalı. `tls-sig-api-v2` ile sunucu tarafında `userSig` üretilir; **secret key hiçbir zaman istemciye gitmez.** ✓
- Kanonik oda kimliği `voiceTrtcRoomId(roomId)` → `voice_room_<id>` hem web hem Flutter'a döner; böylece iki taraf **aynı TRTC odasına** düşer. ✓
- Deterministik `numericUid (userIdToNumericUid)` sayısal kimlik gerektiren SDK yolları için döner. ✓
- `expireTime` env ile (varsayılan 86400 sn / 24 saat) yapılandırılır. **Token yenileme:** Flutter süre dolmadan önce endpoint'i tekrar çağırarak yeni `userSig` alır — sunucu her çağrıda taze imza üretir. ✓

3.2 Oda katılım/ayrılma ve bayat oturum temizliği (chat/rooms/[roomId]/voice)

- **Join:** `voiceSession.upsert (isActive=true, ping güncellenir) + emitMicChanged(...true)` ile web+Flutter'a SSE yayını. ✓
- **Leave:** `updateMany isActive=false + emitMicChanged(...false)`. ✓
- **Rejoin:** `upsert` sayesinde tekrar katılım sorunsuz (aynı `[roomId,userId]` kaydını yeniden aktifleştirir). ✓
- **Bayat oturum temizliği:** Her GET çağrısında `cleanupInactiveSessions` — **30 sn** ping almayan aktif oturumlar `isActive=false` yapılır. Böylece kopan/çöken istemcilerin “hayalet” oturumları temizlenir. ✓
- **Ping (kalp atışı):** GET çağrısında çağıranın `lastPing` değeri güncellenir. ✓
- **Yetki kontrolü:** join işleminde owner / global admin / voice-rol kontrolü yapılır (403 aksi halde). ✓

Sonuç: TRTC yaşam döngüsü (join → leave → rejoin → disconnect → reconnect → token refresh) kod düzeyinde eksiksizdir. Bayat sunucu oturumları temizlenmektedir.

4. PERFORMANS

4.1 İyi optimize edilmiş noktalar — doğrulandı (kod)

- **Canlı yayın listesi** (`video-streams` GET): 10 sn cache (`getCached`), `Promise.all` ile paralel soru, izleyici sayıları **tek** `groupBy` ile toplu alınır (N+1 YOK). ✓
- **Voice room jeton toplamları** (`lib/voice-room-gifts.ts`): tek `groupBy` , N+1 yok. ✓
- **SSE presence:** her poll'de değil 10 sn'de bir sorgulanır. ✓

- **Cache katmanı** (`lib/cache.ts`): TTL'li, Redis-uyumlu API, thundering-herd koruması, otomatik eviction. ✓

4.2 Tespit edilen N+1 / iyileştirme adayları — öneri

- `app/api/messages/route.ts` : her konuşma için ayrı `directMessage.count()` (okunmamış sayısı) yapıyordu (N+1). ✓ **UYGULANDI:** Tek bir `groupBy` (gönderene göre gruplandırma) ile N sorgu **1 sorguya** indirildi; yanıt şekli (`unreadCount`) aynen korundu.
- `.map(async ...)` kalıbı 11 dosyada mevcut; çoğu `Promise.all` ile sarılıdır (kabul edilebilir). Kritik gerçek zamanlı yollarda değildir.

4.3 Gereksiz log

- Sıcak poll döngüsü içinde gereksiz log **yoktur**. SSE stream açılışında bağlantı başına **tek** bir `[SSE] Stream opened` logu vardır (poll başına değil) — hata ayıklama için faydalı, zararsız. TRTC token logu istek başıdır (join/refresh anları — sıcak değil). Kaldırılması marjinal fayda sağlar; davranışsız risk için dokunulmadı.

5. HEDİYE SİSTEMİ

5.1 Hediye motoru (`lib/gift-engine.ts`) — doğrulandı (kod)

- Profesyonel, DB-destekli: `GiftCombo` (kombo), `GiftQueue` (FIFO sıra), `GiftHistory` (log).
- Birleşik `buildGiftReceivedPayload` — tüm render meta ile (animasyon tipi, ekran alanı, süre, koltuk efekti, ses efekti, kombo eşikleri 2/5/10/50/100).
- Tüm motor çağrıları `try/catch` içinde — hediye motoru bir hata verse bile **para akışı etkilenmez**. ✓

5.2 Video + resim veri tamlığı (`lib/gift-render.ts`) — doğrulandı (kod)

Her istemciye gönderilen render meta **eksiksizdir**: `assetUrl`, `fileUrl`, `videoUrl` (yalnızca video ise), `imageUrl`, `thumbnailUrl`, `previewUrl`, `assetFormat` (png/webp/avif/gif/svg/lottie/mp4/webm), `mediaType`, `width`, `height`, `duration` (ms), `mimeType`, `isFullscreen`, `screenPosition`, `displayDurationMs`, ses/müzik url'leri. Video hediyeler için tüm oynatma alanları (`videoUrl`, `width`, `height`, `duration`, `mimeType`) doldurulur. ✓

5.3 Tekil olay / tekrar / sıra — KRİTİK FLUTTER BULGUSU

Her **tek** hediye gönderimi, **aynı gift SSE kanalı** üzerinden **2-3 mesaj** yayınlar:

1. **Legacy ham payload** — `emitChatEvent(roomId, 'gift', {...})`, `engine` alanı **YOK** (web geriye dönük uyumluluk).
2. **Motor gift_received** — `processGiftSend()` → `{ engine:true, event:'gift_received', ... }`.
3. **Motor gift_queue_updated** — `{ engine:true, event:'gift_queue_updated', queueLength, queue }`.

Üçü de istemciye `type:'gift'` olarak ulaşır. Aynı desen `video-streams/[streamId]/gifts` ve `live/gift/send` uçlarında da vardır (legacy `emitStreamEvent` + motor).

- **Bu çift yayın BİLİNÇLİDİR** (web istemcisi legacy'yi, yeni istemciler motoru kullanır). **Kaldırılmadı** (Kural #8 — web'i kırar).
- **Flutter tarafının uygulaması gereken tekilleştirme (deduplication) kuralı:**
- `engine === true` alanına göre ayrıştır.

- Aynı hediye için **yalnızca BİRİNİ** görselleştir: ya legacy payload'ı **ya da** `gift_received` 'i (ikisini birden DEĞİL). Önerilen: motor payload'ını (`engine:true` , `gift_received`) kullan, legacy'yi yok say.
- Motor olaylarını `event` alanına göre yönet: `gift_received` → hediyeyi göster/animasyon; `gift_queue_updated` → sırayı güncelle; `gift_finished` → sıradan çıkar.
- Sıra korunması: aynı tip içinde SSE sırası korunur; kombo/sıra durumu için motorun `queueIndex` / `queue` alanlarına güvenilmelidir.

5.4 Para yolu atomikliği — ✓ UYGULANDI

- `video-streams/[streamId]/gifts` ve `live/gift/send` para akışları `$transaction` kullanır (atomik). ✓
- `chat/rooms/[roomId]/gifts` jeton düşme/ekleme işlemleri de artık atomiktir: gönderen jeton düşümü, alıcı jeton alacağı, oda sahibi komisyonu ve ilgili tüm `jetonTransaction.create` çağrıları tek bir `prisma.$transaction` içine alındı. Kısmi bakiye güncellemesi riski ortadan kalktı.
- **Davranış korundu:** Tutarlar, komisyon oranı ve yanıt şekli aynen bırakıldı; yalnızca işlemler atomik hale getirildi. `processAgencyCommission` (ateşle-ve-unut) ve gelir loglama bilinçli olarak `transaction` dışında tutuldu.
- **Doğrulama:** Değişiklik build/tip kontrolünden geçti (canlı ortamda ayrıca test edilmesi önerilir).

6. VERİTABANI

6.1 İndeksler — doğrulandı (kod), iyi durumda

Sıcak gerçek zamanlı modellerin tümü iyi indekslenmiştir:

- `ChatPresence` : `[roomId, lastSeen]`
- `VoiceSession` : `@@unique([roomId, userId]) + [roomId, [isActive], [lastPing]`
- `GiftQueue` : `[contextId, status, queueIndex]`
- `GiftCombo` : `unique [contextId, senderId, giftTypeId]`
- `GiftHistory` : birden çok `[*, createdAt]`
- `LiveSession` : `[tellerId], [userId], [status], [roomId]`
- `ChatMessage` : `[roomId, createdAt]`
- `RoomSignal` : `[sessionId], [receiverId, processed]`

Değerlendirilen ek indeks: `VoiceSession [roomId, isActive]` bileşik indeksi düşünüldü; ancak mevcut `@@unique([roomId, userId])` zaten `roomId` önekini karşıladığından ve oda başına oturum sayısı küçük olduğundan **marjinal fayda** sağlar. “Önek fazlalığından kaçın” ilkesi gereği **eklenmedi** (paylaşımlı şemaya gereksiz indeks eklemekten kaçınıldı).

6.2 Bağlantı havuzu — doğrulandı (kod), iyi yapılandırılmış

`lib/db.ts` : tekil (singleton) Prisma istemcisi, `connection_limit=5` , `pool_timeout=10` , `connect_timeout=5` , `statement_timeout=5000` . Hot-reload/serverless için global cache. Bildirim oluşturulunca OneSignal push tetikleyen Prisma middleware (fire-and-forget, DB yanıtını bloklamaz). ✓

6.3 İşlemler (transactions)

- `app/api` içinde toplam **33** `$transaction` kullanımı. Para/hediye yollarının çoğu atomiktir (istisna \$5.4'te belirtilen chat-room hediye yolu).

6.4 Kilitler (locks)

- Uzun süre kilit tutan bir desen tespit edilmedi. `statement_timeout=5000` uzun süren sorguları keserek kilit birikimini sınırlar. Kısa idle timeout'lar ile bağlantılar geçici (ephemeral) varsayılır.

7. YAPILAN DEĞİŞİKLİKLER

Kullanıcı onayıyla **iki güvenli, davranış-koruyan (behavior-preserving) değişiklik** uygulandı. Her ikisi de yanıt şekillerini ve iş mantığını aynen korur; build/tip kontrolünden geçmiştir:

1. **Chat-room hediye para yolu atomikleştirildi** (`app/api/chat/rooms/[roomId]/gifts/route.ts`): jeton düşümü/alacak/komisyon hareketleri tek bir `prisma.$transaction` içine alındı. Kısmi bakiye güncellemesi riski ortadan kalktı. (Ayrıntı: §5.4)
2. **Messages N+1 sorgusu giderildi** (`app/api/messages/route.ts`): konuşma başına ayrı `count()` yerine tek `groupBy`. Konuşma sayısı kadar sorgu → 1 sorgu. (Ayrıntı: §4.2)

Uygulanmayan iyileştirmeler (ya mevcut Flutter istemcisini kırar, ya mimari büyük değişikliktir) **onaya sunulan öneriler** olarak §8'de kalıcı risk olarak listelenmiştir (zarf standardizasyonu, Redis pub/sub).

8. DÜZELTİLMİYEN RİSKLER (Kalıcı)

1. **In-memory olay yolu örnek-başınadır** — yatay ölçeklemede SSE olay kaybı riski. Çözüm: Redis pub/sub (büyük değişiklik, onay gerekir).
2. **API yanıt zarfı tutarsızlığı** — ~287 endpoint standart zarf kullanmaz. Mevcut Flutter buna göre yazıldığı için düzeltilmedi; yeni uçlarda `lib/api-response.ts` kullanılmalı.
3. **Poll penceresi içi tipler-arası sıralama garantisi yok** — Flutter kronolojik sırayı payload `timestamp` alanından türetmeli.
4. **Denetim kapsamı**: 487 endpoint'in tamamı canlı ortamda çalıştırılarak test edilmemiştir; kritik gerçek zamanlı/hediye/TRTC/DB yolları kod düzeyinde doğrulanmıştır.

9. FLUTTER TARAFININ YAPMASI GEREKENLER

1. **Hediye tekilleştirme (EN KRİTİK)**: Her hediye 2-3 SSE mesajı üretir. `engine === true` ile ayırıştır; aynı hediye yalnızca bir kez göster (önerilen: motor `gift_received` 'i kullan, legacy'yi yok say). Motor olaylarını `event` alanına göre yönet (`gift_received` / `gift_queue_updated` / `gift_finished`).
2. **Olay sıralaması**: Farklı tipteki SSE olayları arasında mutlak kronolojik sıraya güvenme; gerekiyorsa payload `timestamp` / `createdAt` ile istemcide sırala.
3. **SSE reconnect**: Kopmada son alınan olay id'sini `Last-Event-ID` başlığı veya `?lastEventId=` ile geri gönder — TTL penceresi içindeki kaçırılan olaylar tekrar alınır.
4. **Heartbeat**: 15 sn'de bir : `heartbeat` satırı gelir; bunu bağlantı canlılık göstergesi olarak kullan, timeout'u buna göre ayarla (örn. 30-45 sn).
5. **TRTC token yenileme**: `expireTime` (varsayılan 24 saat) dolmadan `/api/trtc/token` 'ı tekrar çağırıp yeni `userSig` al. Odaya girişte daima dönen `trtcRoomId` (`voice_room_<id>`) değerini kullan — kendi oda id'ini üretme.

6. **Voice ping:** Voice oturumunu canlı tutmak için düzenli olarak (30 sn'den kısa aralıkla) voice GET çağır; aksi halde oturum bayat sayılıp temizlenir.
 7. **Yanıt şekilleri:** Her endpoint'in kendi şekline göre parse et; API genelinde tek tip zarf **varsayma** (dokümantasyondaki `FLUTTER_API_REFERENCE` 'a uy).
-

10. Sonuç

canlifal.com backend'i, Flutter mobil uygulamasıyla çalışmaya **büyük ölçüde hazırdır**. Kritik yollar (gerçek zamanlı akış, TRTC yaşam döngüsü, hediye motoru, veritabanı) sağlam ve optimize durumdadır. chat-room hediye para yolu atomik hale getirildi (§5.4) ve mesaj kutusu N+1 sorgusu giderildi (§4.2). Tam "%100 sertifikasyon" için kalan adımlar: (1) Flutter tarafının §9'daki kuralları uygulaması, (2) ölçekleme planlanıyorsa in-memory olay yolunun Redis pub/sub'a taşınması (§8.1).

§5.4 (para yolu atomikliği) ve §4.2 (N+1) artık **tamamlandı**. Geriye kalan tek koşul §9'daki Flutter tarafı uyarlamalarıdır; bunlar tamamlandığında backend, tek örnekli dağıtımda Flutter ile tam uyumlu kabul edilebilir.